

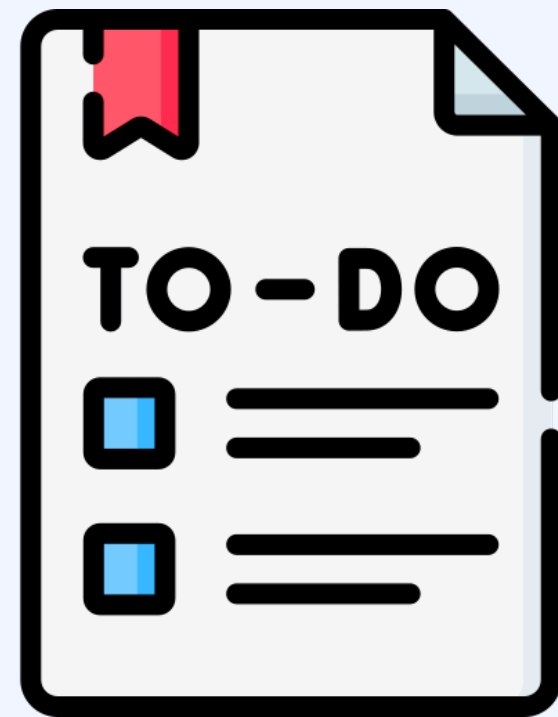
TODO 앱

1 개요 및 학습목표

프로젝트 소개

1.

개요 및 학습목표



TODO 앱

- DI 적용
- Room을 이용해 내부 DB 접근
- Style/Theme 적용

구현에 필요한 기술들

1.

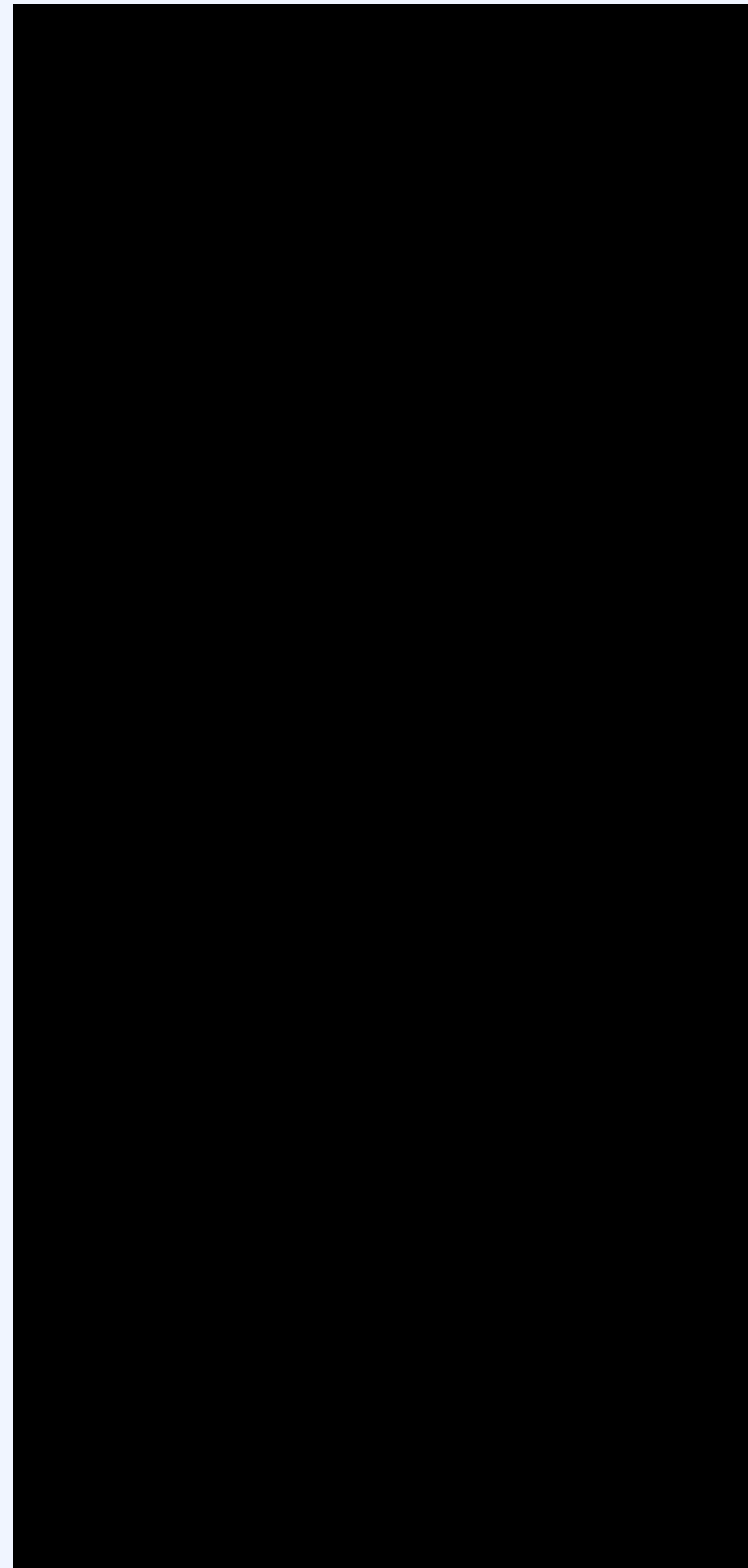
개요 및 학습목표

- MVVM
- Hilt
- Coroutine
- Flow
- Room

완성 예제

1.

개요 및 학습목표



- Hilt를 활용한 DI적용
- Room을 이용한 CRUD 구현
 - 삽입/조회/수정/삭제
- Style 공통화 처리

TODO 앱

2 사용 기술

Room 이란?

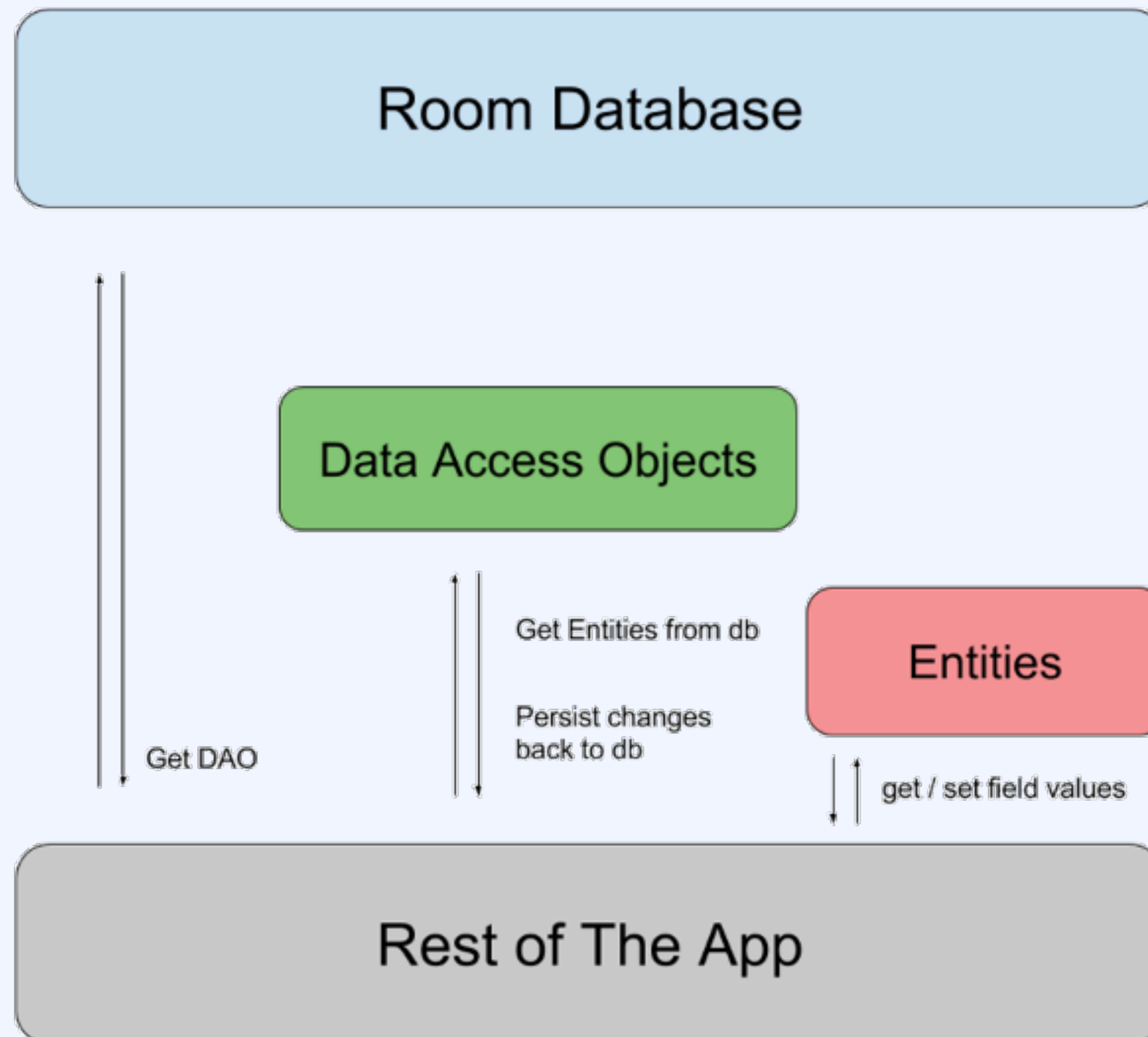
2. Room

AAC(Android Architecture Components)로써 Android 내장 DB에 데이터를 저장하기 위해 사용하는 ORM 라이브러리

SQLite 대신 Room을 사용해야하는 이유

1. 컴파일 도중 SQL에 대한 유효성 검사 가능
2. Schema가 변경될 시 자동으로 업데이트 가능
3. Java 데이터 객체를 변경하기 위해 상용구 코드 없이 ORM 라이브러리를 통해 매핑 가능
4. LiveData와 RX Java를 위한 Observation 생성 및 동작 가능

Room 이란?



구성요소

1. Entity
2. DAO(Data Access Objects)
3. Room DataBase

Entity

2.
Room

```
@Entity(tableName = "Content")
data class ContentEntity(
    @PrimaryKey(autoGenerate: true)
    val id: Int = 0,

    @ColumnInfo
    var content: String,

    @ColumnInfo
    var memo: String? = null
) : Serializable
```


DAO(Data Access Objects)

2.
Room

```
@Dao
interface ContentDao {

    @Query("SELECT * FROM Content")
    fun selectAll(): Flow<List<ContentEntity>>

    @Delete
    suspend fun delete(item: ContentEntity)

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(item: ContentEntity)
}
```

DAO(Data Access Objects)

@Insert 충돌 처리 방식

종류	방식
OnConflictStrategy.ABORT	충돌이 발생할 경우 처리 중단
OnConflictStrategy.FAIL	충돌이 발생할 경우 실패처리
OnConflictStrategy.IGNORE	충돌이 발생할 경우 무시
OnConflictStrategy.REPLACE	충돌이 발생할 경우 덮어쓰기
OnConflictStrategy.ROLLBACK	충돌이 발생할 경우 이전으로 되돌리기

Room DataBase

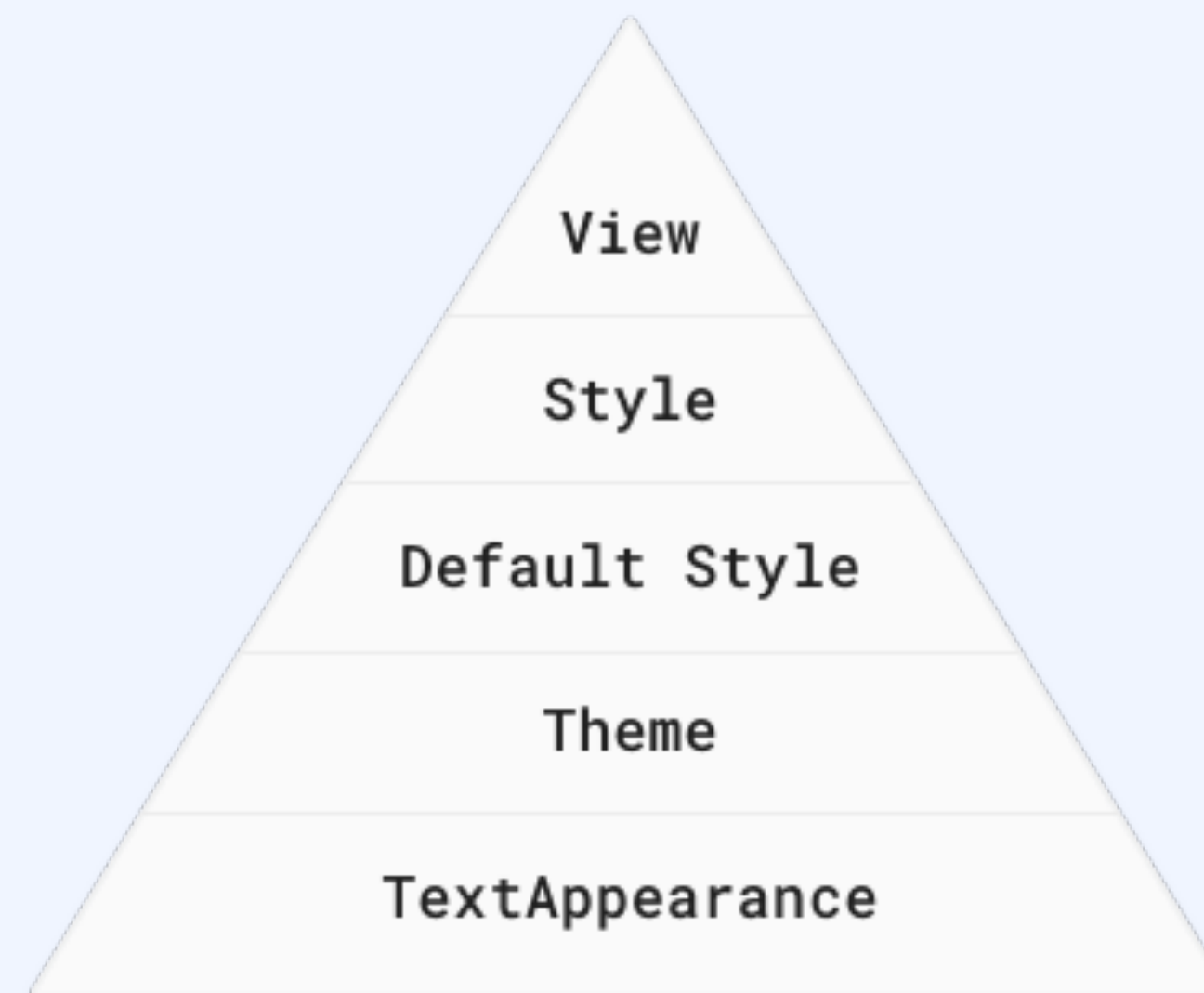
2. Room

```
@Database(entities = [ContentEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {

    abstract fun contentDao() : ContentDao
}
```


Android Styling system

3. Theme



1. View Attributes
2. Styles
3. Default Style
4. Themes
5. TextAppearance